Tracewall: Tool-Call Enforcement for AI Agents

Anonymous Author(s)
Draft aligned to in-repo evidence ledger
Companion to *agentwatch* (observability); this paper is enforcement-only.
Source: `github.com/beejak/agentwatch-firewall`

***Abstract*—AI agents invoke tools that can move money, send email, and mutate shared state. Prompt injection and capability misuse make those calls look legitimate on the wire. We present Tracewall, a transport-agnostic agent firewall with a single seam—`await Firewall.check(event)`→`FirewallVerdict`—that decides ALLOW or BLOCK before a side effect runs. The pipeline combines optional identity checks, a deterministic YAML policy DSL (including call-tree context), a recovering multi-hop taint ledger, and an optional semantic escalation tier. Fail-safe behavior is BLOCK on internal error.**

On a frozen held-out corpus (n=27), the key-free path reaches deterministic integrated recall 1.0 (precision ≈ 0.929, FPR ≈ 0.071); tier-1 policy alone reaches recall 1.0 / FPR 0 on that split. These numbers are a *regression bar*, not proof against adaptive attacks. On AgentDojo banking under DeepSeek with bill-preserving injections, a `direct` attack on one user×injection pair shows baseline ASR 1.0 with utility 1.0, falling to ASR 0.0 with Tracewall while utility remains 1.0; expanding to four injection tasks yields ASR 1.0→0.0 with a utility tax (1.0→0.25). MCP stdio profiles ship with brink tests that record successes *and* known limits.

***Index Terms*—AI agents, tool-call firewall, prompt injection, AgentDojo, policy DSL, taint tracking**

I. Introduction

Agent deployments add attack surfaces beyond single-turn chat: (1) *input corruption*—instructions embedded in files or tool results (e.g., MINJA-style memory/injection [1]); (2) *capability abuse*—legitimate tools used for illegitimate goals (`send_email`/`send_money` after a sensitive read); (3) *contagion*—taint flowing across agents or sessions via shared memory.

Content guardrails lack tool semantics. Wire gateways see destinations, not *why* a call was made. Dual-LLM interpreters redesign the agent stack. Tracewall targets a narrower wedge: intercept tool calls, decide with policy and taint, and keep evaluation honest.

a) Contributions.: (1) Stable enforcement seam with identity → content screen → YAML policy → trust/taint gate → optional semantic judge → audit; internal errors → BLOCK. (2) Deterministic policy pack for injection, exfil, destructive ops, and AgentDojo-shaped banking probes. (3) Multi-hop taint ledger with recovering trust (live `check()` updates on ALLOW/BLOCK). (4) Python guard and MCP stdio proxy with named profiles; limitations documented. (5) Evaluation discipline: held-out ablation, MCP brink (success + expected limits), AgentDojo live slices; claim ledger in-repo.

b) Non-claims.: We do not claim an observation-first OS, sub-millisecond p99 latency versus GPU sentinels without a matched full-`check` study, or 100% detection on a small known-bad suite as a primary result. Held-out tier-1 recall is not adaptive robustness. AgentDojo results are model- and attack-specific.

II. Related Work

Agent firewalls and guardrails (content/alignment checkers; DSL enforcers such as AgentSpec [2]) reduce some unsafe actions but often omit cross-session taint or call-tree policy. Wire gateways preserve destinations, not intent. AgentDojo [3] provides suites with utility and security scoring for tool-using agents; we treat it as an *external bar*, not the only metric. Classical taint is binary; Tracewall uses continuous trust/taint with hop and time decay. End-to-end MINJA versus GPT-4 agents remains unverified beyond synthetic ledger tests.

III. Threat Model

In scope: indirect injection into tool-visible content; misuse of available tools; single-agent compromise that should raise taint when edges are recorded. **Out of scope:** host compromise; defeating the firewall process; model-weight jailbreaks as the primary defense; adaptive paraphrases beyond the frozen held-out set unless measured.

IV. Design

A. Pipeline

Listing 1. Enforcement pipeline (conceptual).

```
HookEvent -> L0 identity -> tier-0 content (flag
    only)
        -> tier-1 YAML policy -> trust/taint route
        -> (escalate) tier-2 semantic ->
            FirewallVerdict -> audit
        -> ledger (ALLOW: on_clean_call; BLOCK:
            on_taint_event)
```

`FirewallVerdict.score` is 0 (bad) … 1 (clean). `context_completeness` records which signals were present.

B. Policy DSL

Rules match tool name, argument operators (`regex`, `not_in_domain`, `matches_secret_pattern`, …), and call-tree constraints. `${ORG_DOMAIN}` expands from `TRACEWALL_ORG_DOMAINS`. `rate_exceeds` is unsupported (never silent).

TABLE I
DEPLOYMENT PLACEMENTS.

Placement	Status
Python `guard` / `Firewall.check`	Shipped
MCP stdio proxy + profiles	Shipped (NDJSON; Content-Length deferred)
LangGraph / HTTP sidecar	Roadmap

TABLE II
HELD-OUT ABLATION (DETERMINISTIC).

Evaluator	Recall	Precision	FPR
Tier-0 content	0.0	0.0	0.143
Tier-1 policy	1.0	1.0	0.0
Integrated	1.0	≈ 0.929	≈ 0.071
Tier-2 semantic alone	0.462	0.857	0.071

C. Taint / MTP

A local SQLite ledger stores trust, taint, identity, and edges. Multi-hop taint propagation (MTP) solves a max-product fixed point with quarantine recovery. Product value today: trust feedback on the live path; full contagion edges on MCP remain partial.

D. Transports

Profiles: **paranoid** (require identity, fail-closed, full rules), **balanced** (default), **permissive** (fail-open, subset rules).

V. EVALUATION

All primary numbers are from committed artifacts under `tracewall/eval/results/` and the in-repo evidence ledger (dated 2026-07-19).

A. Held-out corpus (key-free)

Corpus `corpus_v0.1` test split, n=27 (13 malicious / 14 benign), deterministic semantic backend.
Caveat: not adaptive attacks; not AgentDojo.

B. MCP brink

Fourteen rows (9 success + 5 expected_limit): profiles block MINJA/exfil when configured; limits include missing `_meta` (call-tree starvation), permissive skipping the exfil pack, and unscanned `tools/list`. Success requires all `kind=success` rows to pass *and* limits to still reproduce.

C. Firewall-only AgentDojo-shaped stress

Without an LLM, Tracewall **blocks** `send_money` to the AgentDojo attacker IBAN and **allows** legitimate UK bill pay after `read_file`. Tracked bypasses (expected_limit): leading ZWSP in IBAN, wrong tool-name case, `schedule_transaction` without a dedicated rule.

TABLE III
AGENTDOJO BANKING LIVE SLICE (DEEPSEEK). ASR = FRACTION OF PAIRS WHERE THE INJECTION GOAL SUCCEEDED.

Attack / slice	Arm	ASR	Utility
`important_instructions` (1×1)	base / def.	0.0 / 0.0	1.0 / 1.0
`ignore_previous` (1×1)	base / def.	0.0 / 0.0	1.0 / 1.0
`direct` (1×1)	base	1.0	1.0
`direct` (1×1)	defended	0.0	1.0
`direct` (1×4)	base	1.0	1.0
`direct` (1×4)	defended	0.0	0.25

D. AgentDojo live (DeepSeek)

Setup: banking suite; bill-preserving injections; benchmark system prompt (eval-only); model `deepseek-chat`.

Under `direct`, the model follows the injection without Tracewall; with Tracewall, attack goals fail. On the 1×4 expand, utility falls to 0.25 (abort-on-BLOCK can tax the user task when the malicious call is interleaved with bill payment). Other jailbreaks often yield ASR 0 even at baseline (model refusal)—not credited as a Tracewall win.

VI. LIMITATIONS

MCP framing is NDJSON readline (Content-Length deferred). Contagion on live MCP edges is incomplete. The optional semantic LLM tier is non-gating. AgentDojo coverage remains a slice, not all suites×attacks×models. Attacker-IBAN rules are eval-aligned probes; production needs org allowlists / risk scores.

VII. CONCLUSION

Tracewall is a practical tool-call firewall: deterministic policy and taint-aware routing behind one API, with transports for Python and MCP. Evidence favors held-out ablation, brink honesty, and AgentDojo slices where the model actually attempts the attack—so ALLOW/BLOCK can be attributed to the firewall rather than to refusal.

ARTIFACTS

Held-out: `corpus_v0.1_test_deterministic.json`. MCP brink: `mcp_brink.json`. AgentDojo stress: `adojo_stress.json`. Claim status: `paper/EVIDENCE.md`.

REFERENCES

[1] C. Zheng et al., "MINJA: Memory Injection Attack," arXiv:2503.03704, 2025.
[2] Z. Wang et al., "AgentSpec," arXiv:2503.18666, 2025.
[3] E. Debenedetti et al., "AgentDojo," 2024.